

UNIDAD 2. TEMA 1

Introducción a la Programación Interpretada Multiparadigma

Un lenguaje cuyo programa es ejecutado por un **intérprete**, línea a línea, y que admite múltiples estilos de programación: imperativo, estructurado, orientado a objetos y funcional.

Lenguajes destacados:

 PYTHON

 JAVASCRIPT

 RUBY

 PHP



1. Concepto

¿Qué significa interpretada?

Un lenguaje interpretado ejecuta las instrucciones línea por línea mediante un programa llamado intérprete, sin necesidad de compilar previamente todo el código.

¿Qué significa multiparadigma?

Significa que un mismo lenguaje admite diferentes formas de programar:



Programación Imperativa

Se dan instrucciones paso a paso indicando cómo debe ejecutarse el programa. Ejemplo: usar variables y bucles.



Programación Estructurada

Organiza el código en bloques lógicos: secuencias, condicionales y bucles. Mejora la legibilidad.



Programación Orientada a Objetos

Organiza el código en objetos que combinan datos y comportamiento. Ejemplo: clases e instancias.



Programación Funcional

Trata el código como funciones matemáticas puras, evitando cambios de estado y datos mutables.

📌 Python es uno de los más utilizados porque es sencillo de aprender y muy versátil.

1. Programación Imperativa

Se indica paso a paso qué debe hacer el programa mediante variables, operaciones y decisiones.

```
# Programación Imperativa
```

```
nota1 = 80
```

```
nota2 = 75
```

```
nota3 = 90
```

```
promedio = (nota1 + nota2 + nota3) / 3
```

```
if promedio >= 70:
```

```
    print("Aprobó")
```

```
else:
```

```
    print("Reprobó")
```

```
print("Promedio:", promedio)
```

Características observadas

→ Uso de variables

→ Instrucciones secuenciales

→ Cálculos paso a paso

→ Uso de condiciones (if)

2. Programación Estructurada

El código se organiza en bloques lógicos y funciones para mejorar la claridad.

```
# Programación Estructurada

def calcular_promedio(n1, n2, n3):
    return (n1 + n2 + n3) / 3

def mostrar_resultado(promedio):
    if promedio >= 70:
        print("Aprobó")
    else:
        print("Reprobó")

promedio = calcular_promedio(80, 75, 90)

mostrar_resultado(promedio)
print("Promedio:", promedio)
```

Características observadas

- Uso de funciones
- Código dividido en bloques
- Mayor orden y legibilidad
- Fácil mantenimiento

3. Programación Orientada a Objetos (POO)

Se crean objetos que contienen datos y acciones.

```
# Programación Orientada a Objetos

class Estudiante:

    def __init__(self, nombre, nota1, nota2, nota3):
        self.nombre = nombre
        self.nota1 = nota1
        self.nota2 = nota2
        self.nota3 = nota3

    def calcular_promedio(self):
        return (self.nota1 + self.nota2 + self.nota3) / 3

    def mostrar_resultado(self):
        promedio = self.calcular_promedio()
        if promedio >= 70:
            print(self.nombre, "aprobó")
        else:
            print(self.nombre, "reprobó")
        print("Promedio:", promedio)

estudiante1 = Estudiante("Ericka", 80, 75, 90)
estudiante1.mostrar_resultado()
```

Características observadas

- | | |
|--|---|
|  Uso de clases y objetos |  Los datos y comportamientos están agrupados |
|  Fácil reutilización del código |  Adecuado para programas grandes |

4. Programación Funcional

Se basa principalmente en funciones que reciben datos y devuelven resultados.

```
# Programación Funcional

def calcular_promedio(notas):
    return sum(notas) / len(notas)

def resultado(promedio):
    return "Aprobó" if promedio >= 70 else "Reprobó"

notas = [80, 75, 90]

promedio = calcular_promedio(notas)

print(resultado(promedio))
print("Promedio:", promedio)
```

Características observadas

$f(x)$ Uso de funciones



Las funciones reciben datos y devuelven resultados



Se evita modificar variables innecesariamente



Código más simple y reutilizable

Resumen: Los 4 Paradigmas en Python

Los cuatro programas resuelven exactamente el mismo problema, pero lo hacen siguiendo diferentes estilos de programación. Esto demuestra por qué Python es un lenguaje multiparadigma.

Paradigma	Característica principal
Imperativa	Indica paso a paso qué hacer.
Estructurada	Organiza el programa en bloques y funciones.
Orientada a Objetos	Usa clases y objetos que combinan datos y acciones.
Funcional	Se basa en funciones que reciben datos y devuelven resultados.

📌 Observa que los cuatro programas resuelven exactamente el mismo problema, pero lo hacen siguiendo diferentes estilos de programación. Esto demuestra por qué Python es un lenguaje multiparadigma.

2. Modo Intérprete

Es una forma de ejecutar instrucciones de manera inmediata. El intérprete lee y ejecuta cada línea de código en el momento en que se escribe.

El símbolo `>>>` indica que estamos trabajando directamente en el intérprete de Python.

```
>>> 5 + 3  
8
```

Ventajas del modo intérprete

 Resultados inmediatos

 Fácil aprendizaje

 Permite probar instrucciones rápidamente

 Facilita la detección de errores

Entorno de Programación

El entorno es el espacio donde se desarrolla y ejecuta el programa.



IDLE

Entorno básico incluido con Python, ideal para principiantes.



Visual Studio Code

Editor potente con extensiones para Python y depuración avanzada.



PyCharm

IDE completo especializado en Python con herramientas profesionales.



Jupyter Notebook

Ideal para ciencia de datos, permite mezclar código y texto.

Herramientas que ofrecen:

- Editor de código
- Ejecución de programas
- Resaltado de sintaxis
- Depuración de errores

3. Código Fuente

El código fuente es el conjunto de instrucciones escritas por un programador en un lenguaje de programación para que una computadora realice una tarea determinada.

```
nombre = "Ericka"  
print(nombre)
```

"En este ejemplo, las dos líneas constituyen el código fuente."

Características

 Comprensible para los programadores

 Puede modificarse y actualizarse

 Se almacena en archivos

 Define el comportamiento de un programa

 En Python normalmente se guarda con extensión .py — por ejemplo: programa.py

4. Números en Python

Los números son tipos de datos que permiten realizar operaciones matemáticas dentro de un programa.

Números enteros (int)

No tienen decimales.

```
edad = 20
```

Ejemplos: 10, 25, 100

Números decimales (float)

Contienen parte decimal.

```
precio = 99.50
```

Ejemplos: 3.14, 2.5, 100.75

Operaciones matemáticas básicas

```
a = 10
```

```
b = 5
```

```
print(a + b) # suma → 15
```

```
print(a - b) # resta → 5
```

```
print(a * b) # multiplicación → 50
```

```
print(a / b) # división → 2.0
```

5. Cadenas de Caracteres (str)

Las cadenas de caracteres o strings son secuencias de texto encerradas entre comillas.

```
nombre = "Ericka"  
ciudad = "Heredia"
```

Operaciones con cadenas



Concatenar

Unir textos.

```
nombre = "Ericka"  
apellido = "Hernandez"  
print(nombre + " " + apellido)
```

Resultado: Ericka Hernandez



Obtener longitud

```
texto = "Python"  
print(len(texto))
```

Resultado: 6



Convertir a mayúsculas

```
texto = "python"  
print(texto.upper())
```

Resultado: PYTHON

6. Listas en Python

Las listas son estructuras de datos que permiten almacenar varios elementos dentro de una sola variable.

Características

 Ordenadas

 Modificables

 Permiten diferentes tipos de datos

 Admiten elementos repetidos

01

Crear una lista:

```
frutas = ["Manzana", "Pera", "Uva"]
```

02

Acceder a un elemento:

```
print(frutas[0])
```

→ Manzana

03

Modificar un elemento:

```
frutas[1] = "Banano"
```

→ ["Manzana", "Banano", "Uva"]

04

Agregar / Eliminar:

```
frutas.append("Naranja") # agrega  
frutas.remove("Uva")    # elimina
```

05

Recorrer la lista:

```
for fruta in frutas:  
    print(fruta)
```

→ Manzana, Banano, Naranja

Glosario

1	Intérprete Programa que ejecuta el código fuente línea por línea, sin necesidad de compilarlo previamente.
2	Lenguaje interpretado Lenguaje de programación cuyos programas son ejecutados directamente por un intérprete.
3	Multiparadigma Característica de un lenguaje que permite usar diferentes estilos o paradigmas de programación.
4	Paradigma Estilo o enfoque de programación que define cómo se organiza y escribe el código.
5	Programación imperativa Estilo en el que se indica paso a paso qué debe hacer el programa mediante instrucciones secuenciales.
6	Programación estructurada Organiza el código en bloques lógicos (secuencias, condicionales y bucles) para mejorar la claridad.
7	Programación orientada a objetos (POO) Estilo que organiza el código en clases y objetos que combinan datos y comportamiento.
8	Programación funcional Estilo basado en funciones puras que reciben datos y devuelven resultados, evitando cambios de estado.
9	Código fuente Conjunto de instrucciones escritas por un programador en un lenguaje de programación.
10	Variable Espacio en memoria que almacena un valor que puede cambiar durante la ejecución del programa.
11	String (cadena) Tipo de dato que representa texto, encerrado entre comillas simples o dobles.
12	Lista Estructura de datos que permite almacenar varios elementos ordenados dentro de una sola variable.
13	int Tipo de dato numérico que representa números enteros (sin decimales).
14	float Tipo de dato numérico que representa números con parte decimal.
15	Entorno de programación Espacio o herramienta donde se escribe, ejecuta y depura el código de un programa.